

CineMax

Manuale Tecnico

Sistema di gestione per cinema monosala

Laboratorio Interdisciplinare A — a.a. 2025/2026

Università degli Studi dell'Insubria

Kejsi Xhafaj
Matricola 759934 — Sede VA

Versione 1.0 — 31 agosto 2026

Indice

1	Introduzione e obiettivi	3
2	Installazione	4
2.1	Requisiti di Sistema	4
2.2	Setup ambiente di sviluppo	4
2.3	Compilazione e generazione dell'artefatto	4
2.3.1	Compilazione	4
2.3.2	Creazione del jar	4
2.3.3	Generazione documentazione Javadoc	4
2.3.4	Esecuzione	5
3	Architettura del sistema	6
3.1	Organizzazione in livelli	6
3.1.1	Livello di presentazione	6
3.1.2	Livello di logica applicativa	6
3.1.3	Livello di persistenza	6
3.1.4	Livello del modello	6
3.2	Diagramma delle dipendenze	7
3.3	Elenco delle classi	7
3.4	Sequenza di inizializzazione	7
4	Modello del dominio	9
4.1	Film e Genere	9
4.2	Proiezione e Palinsesto	9
4.3	Utente e Ruolo	11
4.4	Prenotazione	11
5	Strutture dati utilizzate	13
6	Persistenza e formato dei file	15
6.1	Risoluzione dei percorsi	15
6.2	Formato dei tre file CSV	15
6.3	Parsing e serializzazione	16
6.4	Retrocompatibilità e migrazione	17
6.5	Riallineamento dei dati derivati	17
7	Scelte algoritmiche	19
7.1	Controllo di sovrapposizione temporale	19
7.2	Generazione dei codici univoci	19
7.3	Ricerca e filtraggio	20
8	Sicurezza e gestione delle credenziali	22

9	Gestione degli errori e delle eccezioni	25
9.1	Gerarchia delle eccezioni	25
9.2	Strategia di caricamento	25
9.3	Propagazione degli errori di salvataggio	26
9.4	Riferimenti orfani	26
10	Regole di dominio implementate	27
10.0.1	Regole - Proiezioni	27
10.0.2	Regole - Prenotazioni	27
10.0.3	Regole - Utenti	27
11	Pattern e principi di progettazione adottati	29
11.1	Separazione in livelli	29
11.2	Facade	29
11.3	Dependency injection tramite costruttore	29
11.4	Classi di utilità	29
11.5	Overloading per la leggibilità della validazione	29
12	Limiti della soluzione sviluppata	31
13	Sitografia e bibliografia	32

Capitolo 1

Introduzione e obiettivi

CineMax è un'applicazione Java a interfaccia testuale per la gestione del palinsesto e delle prenotazioni di un cinema monosala con capienza di 200 posti. Il documento descrive l'architettura del sistema, le strutture dati impiegate, il formato di persistenza, le scelte algoritmiche e le motivazioni tecniche che le hanno determinate. La documentazione Javadoc generata dai sorgenti, disponibile in `doc/javadoc/`, costituisce parte integrante di questo manuale e ne rappresenta il riferimento di dettaglio a livello di singola classe e metodo. Gli obiettivi progettuali perseguiti sono stati:

1. Separazione delle responsabilità fra rappresentazione dei dati, logica applicativa, persistenza e interazione con l'utente, in modo che ciascun livello sia modificabile in modo indipendente.
2. Validazione centralizzata e sistematica, applicata sia in costruzione sia in modifica, così che nessun oggetto possa esistere in uno stato incoerente.
3. Robustezza rispetto ai dati esterni: i file CSV sono modificabili manualmente e possono contenere errori; il sistema deve degradare in modo controllato anziché terminare.
4. Conformità alle specifiche del progetto.

Capitolo 2

Installazione

2.1 Requisiti di Sistema

Componente	Versione
JDK	Java Development Kit (JDK) versione 21
Librerie esterne	Nessuna
Codifica sorgenti	UTF-8

Il progetto utilizza esclusivamente le API standard della piattaforma Java SE.

2.2 Setup ambiente di sviluppo

Il progetto è indipendente dall'IDE. Importandolo in IntelliJ IDEA, Eclipse o VS Code è sufficiente indicare `src` come cartella dei sorgenti.

2.3 Compilazione e generazione dell'artefatto

2.3.1 Compilazione

Per compilare il progetto si consiglia il seguente comando:

```
javac --release 21 -encoding UTF-8 -d build src/cinemax/*.java
```

2.3.2 Creazione del jar

Per creare l'eseguibile invece si consiglia il seguente comando:

```
jar --create --file bin/CineMax.jar --main-class  
cinemax.CineMax -C build .
```

L'opzione `main-class` scrive l'attributo `Main-Class` nel manifest, rendendo l'archivio direttamente eseguibile con `java -jar`.

2.3.3 Generazione documentazione Javadoc

Per generare la Javadoc è consigliato il seguente comando:

```
javadoc -private -encoding UTF-8 -docencoding UTF-8 -charset  
UTF-8 -author -version -d doc/javadoc -sourcepath src cinemax
```

Le opzioni non sono arbitrarie:

- `-private` è necessaria perché tutti gli attributi delle classi sono dichiarati `private` e le classi di utilità `Sicurezza` e `Validatore` hanno visibilità di package; senza questa opzione non comparirebbero nella documentazione generata;
- `-author` e `-version` sono necessarie perché Javadoc, per impostazione predefinita, ignora i tag `@author` e `@version` pur essendo presenti in tutte le classi;
- le tre opzioni di codifica garantiscono che i caratteri accentati siano visualizzati correttamente nell'HTML prodotto.

2.3.4 Esecuzione

Per eseguire l'eseguibile, si usa il seguente comando:

```
java -jar bin/CineMax.jar
```

Capitolo 3

Architettura del sistema

3.1 Organizzazione in livelli

L'applicazione è organizzata in quattro livelli logici, tutti collocati nel package `cinemax`. La separazione fra livelli è realizzata a livello di responsabilità delle classi.

3.1.1 Livello di presentazione

`InterfacciaTerminale` e `CineMax`. Si occupa esclusivamente della raccolta degli input e della formattazione degli output. Non contiene regole di dominio: ogni decisione di validità è delegata ai livelli sottostanti, che rispondono lanciando eccezioni gestite tramite try-catch.

3.1.2 Livello di logica applicativa

`GestoreUtenti`, `GestoreProiezioni`, `GestorePrenotazioni`. Ciascun gestore si occupa di una specifica area funzionale, verifica i permessi dell'utente che richiede le operazioni, applica le regole di dominio e richiede il salvataggio dei dati al termine di ogni operazione che modifichi lo stato.

3.1.3 Livello di persistenza

`GestoreDati`. Centralizza tutta l'interazione con il file system, mantiene in memoria le collezioni condivise ed espone i metodi di ricerca per identificativo.

3.1.4 Livello del modello

`Film`, `Genere`, `Proiezione`, `Palinsesto`, `Prenotazione`, `Utente`, `Ruolo`. Contiene lo stato del dominio e le regole di validità che ciascuna entità può verificare autonomamente.

3.2 Diagramma delle dipendenze

3.3 Elenco delle classi

Classe	Tipo	Responsabilità
CineMax	classe	Punto di ingresso; istanzia i gestori e avvia la TUI
InterfacciaTerminale	classe	Menu, raccolta input, formattazione output, paginazione
GestoreUtenti	classe	Registrazione, autenticazione, modifica profilo
GestoreProiezioni	classe	Ricerca proiezioni, controllo sovrapposizioni
GestorePrenotazioni	classe	Ricerca delle prenotazioni, verifica età e disponibilità
GestoreDati	classe	Lettura/scrittura CSV, collezioni in memoria, ricerche per codice
Palinsesto	classe	Collezione delle proiezioni programmate
Proiezione	classe	Film + collocazione temporale + prezzo + occupazione
Film	classe	Dati descrittivi opera cinematografica
Prenotazione	classe	Associazione utente-proiezione con numero di biglietti
Utente	classe	Anagrafica e credenziali
Genere	enum	Generi cinematografici
Ruolo	enum	Ruoli e relativi permessi
Sicurezza	classe utilità	Hashing, salt, codifica Base64
Validatore	classe utilità	Controlli su stringhe e riferimenti nulli
PermessiMancantiException	eccezione	Operazione non consentita dal ruolo
PostiEsauritiException	eccezione	Capienza della sala superata
EtaMinimaConsentitaException	eccezione	Età minima non rispettata
PrenotazioneInesistenteException	eccezione	Prenotazione non trovata
SovrapposizioneProiezioneException	eccezione	Conflitto temporale nel palinsesto
ProiezioneConPrenotazioniException	eccezione	Modifica/eliminazione bloccata da prenotazioni

3.4 Sequenza di inizializzazione

```
1 public static void main(String[] args) {
2     try {
3         GestoreDati gestoreDati = new GestoreDati();
4         GestoreUtenti gestoreUtenti = new GestoreUtenti(
5             gestoreDati);
6         GestoreProiezioni gestoreProiezioni = new
7             GestoreProiezioni(gestoreDati);
8         GestorePrenotazioni gestorePrenotazioni = new
9             GestorePrenotazioni(gestoreDati);
10        InterfacciaTerminale tui = new InterfacciaTerminale();
11        tui.avvia(gestoreUtenti, gestoreProiezioni,
            gestorePrenotazioni);
    } catch (Exception e) {
        System.err.println("Impossibile avviare CineMax: " + e.
            getMessage());
        System.err.println("Verifica che la cartella 'data' sia
            raggiungibile e che i file CSV siano leggibili e
            scrivibili.");
    }
}
```



```
12     }  
13 }
```

Il **GestoreDati** è ciò che viene costruito per primo perché il suo costruttore esegue l'intero caricamento dei dati; i tre gestori ricevono la stessa istanza, condividendo così le collezioni in memoria. Questa condivisione è essenziale: quando **GestorePrenotazioni** modifica i posti occupati di una proiezione, la modifica è immediatamente visibile a **GestoreProiezioni**, perché entrambi operano sullo stesso oggetto **Proiezione** e non su copie distinte.

Capitolo 4

Modello del dominio

Il principio applicato in tutto il modello è quello dell'invariante di classe: ogni oggetto valida i propri dati sia nel costruttore sia in ciascun metodo di modifica, in modo che non sia possibile portarlo in uno stato incoerente. La validazione è realizzata da metodi privati, richiamati sia dal costruttore sia dai setter, così da evitare duplicazioni di codice.

4.1 Film e Genere

Film incapsula i dati descrittivi dell'opera. Le regole di validità:

Attributo	Vincolo
titolo	non nullo, non vuoto
genere	non nullo
regista	non nullo, non vuoto
anno	fra 1885 e anno corrente +10
durata	strettamente maggiore di 0
etaMinima	maggiore o uguale a 0

Il limite superiore dell'anno è calcolato dinamicamente su `LocalDate.now().GetYear()` anziché essere una costante: così la regola resta valida nel tempo senza richiedere continue modifiche di anno in anno.

Genere è realizzato come enumerazione, rende impossibile la presenza di varianti ortografiche nei dati e consente metodi di confronto più solidi.

Ogni costante associa il proprio nome italiano a quello inglese:

```
1 public enum Genere {
2     AZIONE("Action"),
3     AVVENTURA("Adventure"),
4     ANIMAZIONE("Animation"),
5     BIOGRAFICO("Biography")
6     ....
}
```

Il metodo `daStringa` accetta entrambe le forme, senza distinzione fra maiuscole e minuscole. Questa doppia corrispondenza esiste per la compatibilità col file proiezioni.csv originale che usa nomenclatura inglese.

4.2 Proiezione e Palinsesto

Proiezione è l'entità più complessa del codice data la presenza di orario di inizio, fine e durata del film associato che devono essere controllati per evitare che ci siano situazioni irregolari (o senza senso) che porterebbero a errori non previsti.

Il controllo è effettuato tramite il metodo seguente:

```

1      private static void validaCoerenzaOrari(LocalDateTime inizioProiezione,
2      LocalDateTime fineProiezione, int durataFilm) {
3          Validatore.validaOggetto(inizioProiezione, "L'orario di inizio
4          proiezione non è valido.");
5          Validatore.validaOggetto(fineProiezione, "L'orario di fine
6          proiezione non è valido.");
7          if (inizioProiezione.isAfter(fineProiezione)) {
8              throw new IllegalArgumentException("L'inizio della
9              proiezione non può essere dopo la sua stessa fine.")
10             ;
11         }
12         if (fineProiezione.isBefore(inizioProiezione.plusMinutes(
13             durataFilm))) {
14             throw new IllegalArgumentException("Il film dura " +
15                 durataFilm + " minuti. La fine della proiezione non
16                 consente la sua visione intera.");
17         }
18     }
19 }

```

Il metodo è statico e opera esclusivamente sui parametri, senza accedere ai campi dell'istanza. Questa scelta risolve un problema: essendo i parametri omonimi dei campi, un accesso accidentale allo stato corrente anziché al valore proposto produrrebbe una validazione fatta sui dati sbagliati. Dichiarando il metodo statico, tale errore diventa impossibile per costruzione. Tutti i punti di ingresso, i due costruttori, i setter degli orari e il setter del film, convergono su questo unico metodo, garantendo che la regola sia sempre la stessa.

La modifica di una proiezione però presenta un problema, durata e orari sono interdipendenti e modificarli separatamente attraverso setter distinti permetterebbe di creare stati incoerenti intermedi. Ad esempio, allungare la durata del film senza toccare gli orari produrrebbe una proiezione la cui fine precede il termine della pellicola.

Il metodo seguente risolve il problema calcolando lo stato finale e applicandolo solo se coerente:

```

1      public void aggiornaDurataEOraari(int nuovaDurata, LocalDateTime
2      nuovoInizio, LocalDateTime nuovaFine) {
3          LocalDateTime inizioFinale = (nuovoInizio != null) ? nuovoInizio
4          : this.inizioProiezione;
5          LocalDateTime fineFinale = (nuovaFine != null) ? nuovaFine :
6          this.fineProiezione;
7          int durataFinale = (nuovaDurata > 0) ? nuovaDurata : this.
8          filmProiettato.getDurata();
9          validaCoerenzaOrari(inizioFinale, fineFinale, durataFinale);
10         if (nuovaDurata > 0) {
11             this.filmProiettato.setDurata(nuovaDurata);
12         }
13         this.inizioProiezione = inizioFinale;
14         this.fineProiezione = fineFinale;
15     }
16 }

```

I valori nulli indicano "campo invariato". La validazione precede qualsiasi assegnazione: se lo stato finale non è coerente il metodo lancia un'eccezione prima di aver modificato qualsiasi cosa, e l'oggetto rimane esattamente com'era.

Per questa ragione `gestoreProiezioni.modificaProiezione` invoca questo metodo prima delle altre modifiche: essendo l'operazione con il maggior numero di vincoli è quella con maggiore probabilità di fallire, e collocarla in testa garantisce che un eventuale fallimento avvenga a stato ancora intatto.

Capienza della sala. La costante `POSTI_SALA = 200` è dichiarata in `Proiezione` ed esposta tramite il metodo statico `getPostiSala()`, utilizzando anche da `Prenotazione` e da

GestorePrenotazioni. Il valore è quindi definito in un unico punto: modificare la capienza della sala dunque richiede solo un'unica modifica.

Palinsesto incapsula la collezione delle proiezioni. Espone `getListaProiezioni()` come `Collections.unmodifiableList`, impedendo ai chiamanti di aggiungere o rimuovere elementi bypassando i metodi `aggiungiProiezione` e `rimuoviProiezione`, che effettuano la validazione.

4.3 Utente e Ruolo

Utente mantiene anagrafica e credenziali. Non memorizza mai la password in chiaro, ma soltanto tre elementi che ne permettono la verifica: l'hash, il salt e il numero di iterazioni utilizzate per produrlo.

Vincoli principali:

Attributo	Vincolo
nome, cognome	non vuoti, massimo 50 caratteri
username	solo lettere minuscole e cifre, massimo 50 caratteri
luogoDomicilio	non vuoto, massimo 100 caratteri
iterazioniHashing	almeno 600.000
dataNascita	facoltativa; se è presente, non futura e non antecedente al 1850

La restrizione dello username a lettere minuscole e cifre elimina alla radice due categorie di problemi: il rischio di creare username con caratteri invisibili rendendo la gestione difficoltosa e l'ambiguità di confronto fra maiuscole e minuscole. Costruttori e setter applicano comunque `trim().toLowerCase()` di base, per prevenzione.

Ruolo è un'enumerazione a tre valori. L'appartenenza è verificata tramite i metodi `isProiezionista()`, `isCliente()` e `isBigliettaio()`, che incapsulano il confronto ed evitano la sua ripetizione in ogni gestore.

Gestione della data di nascita facoltativa. Il metodo `getAnniUtente()` restituisce 0 quando l'attributo è assente. Questa convenzione di per sé produrrebbe un comportamento fuorviante: per esempio un utente senza data di nascita risulterebbe indistinguibile da un neonato e riceverebbe il messaggio "non hai l'età minima" per qualsiasi film con restrizioni.

Per questo **GestorePrenotazioni** distingue esplicitamente i due casi prima di applicare il confronto:

```
1      if (proiezioneDaAggiornare.getFilmProiettato().getEtaMinima() > 0 &&
2          utenteCorrente.getDataNascita() == null) {
3          throw new EtaMinimaConsentitaException("Questo film ha un'età
4              minima di " + proiezioneDaAggiornare.getFilmProiettato().
                getEtaMinima() + " anni:" +
                " inserisci la tua data di nascita per prenotare.");
        }
```

L'utente così riceve un messaggio che indica l'azione corretta da compiere per risolvere la situazione.

4.4 Prenotazione

Prenotazione realizza l'associazione fra un utente e una proiezione. Il collegamento non avviene tramite riferimenti diretti agli oggetti ma tramite i loro identificativi: `usernameUtente` e `codiceProiezione`.

Questa scelta è motivata dalla persistenza. Un riferimento a oggetto non è serializzabile su CSV in modo naturale e richiederebbe troppe complicazioni. Utilizzando gli identificativi, ciascun file è leggibile in modo indipendente e la risoluzione del riferimento avviene su richiesta tramite i metodi di ricerca di **GestoreDati**.

Gli attributi `codiceUnivoco` e `usernameUtente` sono dichiarati `final`: identificano la prenotazione e il suo intestatario e non hanno ragione di cambiare durante il ciclo di vita dell'oggetto. Restano modificabili `codiceProiezione` e `numeroBiglietti`.

Capitolo 5

Strutture dati utilizzate

Collezione	Implementazione	Contenuto
Palinsesto.listaProiezioni	LinkedList<Proiezione>	Tutte le proiezioni programmate
GestoreDati.listaUtente	LinkedList<Utente>	Tutti gli utenti registrati
GestoreDati.listaPrenotazioni	LinkedList<Prenotazione>	Tutte le prenotazioni effettuate

Il motivo per cui è stato scelto `LinkedList` è l'utilizzo caratterizzato da inserimenti e rimozioni frequenti e da accessi che avvengono sempre per scansione completa, cioè ricerche per codice, filtrati per criteri multipli, e mai per indice.

In questo modo `LinkedList` offre rimozione in tempo costante una volta individuato il nodo e non richiede riallocazioni dell'array di supporto al crescere della dimensione. L'assenza di accesso casuale non costituisce un limite, poiché nessun punto del codice accede agli elementi per posizione: la paginazione dei risultati opera su liste immutabili già create dai flussi di ricerca, non sulle collezioni sottostanti. **Immutabilità delle viste.** Tutti i getter delle collezioni restituiscono `Collections.unmodifiableList(...)`. Ogni tentativo di modifica strutturale è dunque proibito, costringendo l'uso dei metodi dedicati all'aggiunta e rimozione degli elementi in modo sicuro.

Va precisato che si tratta di immutabilità superficiale: la lista non è modificabile, ma gli oggetti in essa restano comunque mutabili. **Uso della Stream API.** Le operazioni di ricerca e filtraggio sono realizzate con flussi:

```
1 public List<Proiezione> cercaProiezione(String titolo, Genere genere, LocalDate
   dataInizio, LocalDate dataFine, Double prezzoMassimo, Double prezzoMinimo)
   {
2     return gestoreDati.getPalinsesto().getListaProiezioni().stream()
3     .filter(p -> titolo == null || titolo.isBlank() || p.getFilmProiettato()
4     .getTitolo().toLowerCase().contains(titolo.toLowerCase()))
5     .filter(p -> genere == null || p.getFilmProiettato().getGenere() ==
6     genere)
7     .filter(p -> dataInizio == null || !p.getInizioProiezione().toLocalDate
8     ().isBefore(dataInizio))
9     .filter(p -> dataFine == null || !p.getFineProiezione().toLocalDate().
10    isAfter(dataFine))
11    .filter(p -> prezzoMassimo == null || p.getPrezzoBiglietto() <=
12    prezzoMassimo)
13    .filter(p -> prezzoMinimo == null || p.getPrezzoBiglietto() >=
14    prezzoMinimo)
15    .toList();
16 }
```

La forma dichiarativa esprime il criterio di filtro dei parametri opzionali in modo uniforme: ciascun predicato restituisce `true` quando il filtro corrispondente non è stato specificato, ottenendo la combinazione dei criteri. Il metodo `toList()` restituisce una lista immutabile, coerente con

il resto del programma.

Tipi wrapper per i parametri opzionali. I filtri numerati usano Double anziché double per poter distinguere "filtro non specificato" da "filtro con valore zero", distinzione impossibile con un tipo primitivo.

Capitolo 6

Persistenza e formato dei file

6.1 Risoluzione dei percorsi

```
private static final Path CARTELLA_DATA = Files.exists(Paths.get("data"))
    ? Paths.get("data") : Paths.get("../", "data");
```

La risoluzione è relativa alla directory di lavoro e prevede due tentativi: prima `./data`, poi `../data`. Ciò rende l'applicazione eseguibile sia dalla radice del progetto sia dalla cartella `bin`, che è la collocazione del file JAR.

L'uso di `java.nio.file.Path` e del metodo `resolve` garantisce la portabilità: il separatore di percorso per il sistema operativo in cui viene eseguito il programma è applicato automaticamente, senza concatenazioni manuali di stringhe.

6.2 Formato dei tre file CSV

Tutti i file utilizzano la virgola come separatore, hanno una riga di intestazione e sono codificati in UTF-8.

`proiezioni.csv` - 11 campi

#	Campo	Formato
0	codice_univoco	UUID
1	data_ora_inizio_proiezione	yyyy-MM-dd HH:mm:ss
2	data_ora_fine_proiezione	yyyy-MM-dd HH:mm:ss
3	titolo_film	testo
4	genere	nome costante enum
5	regista	testo
6	anno	intero
7	durata_minuti	intero
8	eta_minima	intero
9	prezzo_biglietto	decimale
10	posti_prenotati	intero

`utenti.csv` - 9 campi

#	Campo	Formato
0	nome	testo
1	cognome	testo
2	username	minuscolo alfanumerico
3	password_hash	Base64
4	salt	Base64
5	iterazioni_hashing	intero
6	luogo_domicilio	testo
7	ruolo	nome costante enum
8	data_nascita	yyyy-MM-dd, oppure vuoto

prenotazioni.csv - 4 campi

#	Campo	Formato
0	codice_univoco	7 caratteri
1	username_utente	riferimento a utenti.csv
2	codice_proiezione	riferimento a proiezioni.csv
3	numero_biglietti	intero

Il campo `data_nascita` è l'unico che può essere vuoto, essendo facoltativo. La sua presenza in ultima posizione ha una conseguenza tecnica trattata nel paragrafo successivo.

6.3 Parsing e serializzazione

La gestione del formato CSV è affidata a tre metodi privati di `GestoreDati`: **Separazione dei campi in lettura**

```

1 private String[] separaRigaCSV(String linea, boolean mantieniVuoti) {
2     String[] campi = linea.split(DIVISORE_CSV + "
    (?=(?:[^\"]*" * "[^\"]*" * "\" * [^\"]*" * "$)", mantieniVuoti ? -1 : 0);
3     for (int i = 0; i < campi.length; i++) {
4         campi[i] = pulisciCampoCSV(campi[i]);
5     }
6     return campi;
7 }

```

L'espressione regex separa sulle virgole applicando un cosiddetto lookahead che conta le virgolette rimanenti fino a fine riga: la separazione avviene solo se il numero di virgolette successive è pari, ossia solo se la virgola si trova al di fuori di un campo protetto. Una virgola all'interno di un titolo racchiuso fra virgolette non produce quindi una separazione.

Il parametro `mantieniVuoti` controlla il secondo argomento di `split`. Con il valore predefinito, `split` elimina i campi vuoti finali; il file degli utenti viene pertanto letto con `-1`, poiché la data di nascita assente produce un ultimo campo vuoto che deve essere conservato per non alterare la numerazione degli indici. Gli altri due file non hanno campi finali facoltativi e usano il comportamento predefinito.

Rimozione della protezione

```

1 private String pulisciCampoCSV(String campo) {
2     String risultato = campo.trim();
3     if (risultato.length() >= 2 && risultato.startsWith("\"") && risultato.
        endsWith("\"")) {
4         risultato = risultato.substring(1, risultato.length() - 1).
            replace("\"\"", "");
5     }
6     return risultato;
7 }

```

Le virgolette esterne sono rimosse e le virgolette raddoppiate interne sono ripristinate nella forma singola.

Protezione in scrittura

```
1 private String proteggiCampoCSV(String campo) {
2     if (campo == null) {
3         return "";
4     }
5     if (campo.contains(DIVISORE_CSV) || campo.contains("\"") ||
6         campo.contains("\n") || campo.contains("\r")) {
7         return "\"" + campo.replace("\"", "\\\"") + "\"";
8     }
9     return campo;
}
```

Il campo è racchiuso fra virgolette solo se contiene caratteri che romperebbero il formato, mantenendo il file leggibile nel caso ordinario. Un titolo come **Ammore e malavita**, il film viene scritto come "Ammore e malavita, il film" e riletto correttamente.

Tutti i campi testuali sono protetti in scrittura. I campi numerici e le costanti enum non lo richiedono, non potendo contenere caratteri problematici per costruzione.

6.4 Retrocompatibilità e migrazione

Il file `proiezioni.csv` iniziale adotta un formato a 8 campi, privo di identificativo univoco, di orario di fine e del conteggio dei posti occupati. Il caricamento riconosce entrambi i formati sulla base del numero dei campi presenti.

```
1 try {
2     if (datiProiezione.length == 8) {
3         ...
4         fileDaAggiornare = true;
5     } else if (datiProiezione.length == 11) {
6         ...
7     }
}
```

Al termine del caricamento, se è stata letta almeno una riga nel formato originale, il file viene riscritto nel formato esteso:

```
1 if (fileDaAggiornare) {
2     try {
3         salvaProiezioniSuCSV();
4     } catch (IllegalStateException e) {
5         System.err.println("Impossibile aggiornare proiezioni.
6             csv al formato nuovo: " + e.getMessage());
7     }
}
```

La migrazione avviene una sola volta, al primo avvio sul dataset originale. L'eccezione è intercettata localmente perché ci si trova all'interno del costruttore di `GestoreDati`: propagarla impedirebbe l'avvio dell'applicazione per un'operazione che non è indispensabile al funzionamento. Il sistema opera anche su un file in sola lettura, limitandosi a segnalare l'impossibilità di aggiornarlo.

6.5 Riallineamento dei dati derivati

L'attributo `postiPrenotati` di `Proiezione` è un dato derivato: il suo valore corretto è per definizione la somma dei biglietti di tutte le prenotazioni che fanno riferimento a quella proiezione. La sua persistenza costituisce quindi una denormalizzazione, adottata per evitare di ricalcolare

la somma a ogni visualizzazione di una proiezione, operazione che richiederebbe una scansione completa delle prenotazioni.

Ogni denormalizzazione introduce il rischio di divergenza fra il dato memorizzato e la sua definizione. Nel caso specifico, una riga di `prenotazioni.csv` scartata in fase di caricamento per un errore di validazione lascerebbe i posti conteggiati senza alcuna prenotazione corrispondente. Il metodo `allineaPostiPrenotati()`, invocato come ultima operazione del costruttore, ricostruisce il valore derivato dalla sua definizione:

```
1 private void allineaPostiPrenotati() {
2     for (Proiezione p : palinsesto.getListaProiezioni()) {
3         int totale = 0;
4         for (Prenotazione pr : listaPrenotazioni) {
5             if (pr.getCodiceProiezione().equalsIgnoreCase(p.
6                 getCodiceUnivoco())) {
7                 totale += pr.getNumeroBiglietti();
8             }
9         }
10        if (totale != p.getPostiPrenotati()) {
11            try {
12                p.setPostiPrenotati(totale);
13                System.err.println("Posti prenotati riallineati
14                    per la proiezione " + p.getCodiceUnivoco() +
15                    ".");
16            } catch (PostiEsauritiException e) {
17                System.err.println("Prenotazioni incoerenti per
18                    la proiezione " + p.getCodiceUnivoco() + ":
19                    " + e.getMessage());
20            }
21        }
22    }
23 }
```

Il costo è quadratico nel numero di proiezioni per prenotazioni, ma viene sostenuta una sola volta all'avvio. Il beneficio è la garanzia che il numero di posti liberi mostrato all'utente sia sempre coerente con le prenotazioni effettivamente esistenti, indipendentemente da eventuali manipolazioni manuali dei file.

L'operazione registra su `System.err` ogni riallineamento effettuato, così che un'eventuale incoerenza nei dati sia visibile a chi gestisce il sistema.

Capitolo 7

Scelte algoritmiche

7.1 Controllo di sovrapposizione temporale

Trattandosi di un cinema monosala, due proiezioni non possono occupare simultaneamente la sala. Il controllo è realizzato dal metodo `controllaSovrapposizione` di `GestoreProiezioni`:

```
1 private void controllaSovrapposizione(LocalDateTime nuovoInizio, LocalDateTime
   nuovaFine, String codiceDaIgnorare) {
2     Validatore.validaOggetto(nuovoInizio, "L'orario di inizio per il
       controllo della sovrapposizione non può essere nullo.");
3     Validatore.validaOggetto(nuovaFine, "L'orario di fine per il controllo
       della sovrapposizione non può essere nullo.");
4     for (Proiezione esistente : gestoreDati.getPalinsesto().
       getListaProiezioni()) {
5         if (esistente.getCodiceUnivoco().equalsIgnoreCase(
           codiceDaIgnorare)) {
6             continue;
7         }
8         LocalDateTime inizioEsistente = esistente.getInizioProiezione();
9         LocalDateTime fineEsistente = esistente.getFineProiezione();
10        if (nuovoInizio.isBefore(fineEsistente) && nuovaFine.isAfter(
            inizioEsistente)) {
11            throw new SovrapposizioneProiezioneException("
                Impossibile procedere: l'orario si sovrappone
                temporalmente con la seguente proiezione: \n" +
                    esistente);
12        }
13    }
14 }
```

La condizione implementa il criterio classico di intersezione fra intervalli. L'uso di disuguaglianze strette rende ammissibile le proiezioni contigue: uno spettacolo che termina alle 22:00 e il successivo che inizia esattamente alle 22:00 non generano conflitto.

Il parametro `codiceDaIgnorare` consente di escludere dal confronto una specifica proiezione. E' indispensabile in fase di modifica: senza questa esclusione una proiezione risulterebbe sempre sovrapposta a se stessa, rendendo impossibile qualsiasi modifica di orario.

La complessità è lineare nel numero di proiezioni programmate.

7.2 Generazione dei codici univoci

Il sistema impiega due strategie distinte per esigenze diverse. `Proiezioni - UUID` completo:

```
1 public Proiezione(LocalDateTime inizioProiezione, LocalDateTime fineProiezione,
   Film filmProiettato, double prezzoBiglietto, int postiPrenotati) {
2     this(UUID.randomUUID().toString(), inizioProiezione, fineProiezione,
       filmProiettato, prezzoBiglietto, postiPrenotati);
}
```

```
3 }
```

Il codice non è mai digitato manualmente dall'utente, che seleziona sempre la proiezione da un elenco. Non essendoci vincoli di leggibilità, si adotta un UUID completo, la cui probabilità di collisione è trascurabile e non richiede alcuna verifica.

Prenotazioni - codice breve con verifica esplicita:

```
1      String codicePrenotazioneNuovo = null;
2      boolean codiceOccupato = true;
3      while (codiceOccupato) {
4          codicePrenotazioneNuovo = UUID.randomUUID().toString().substring
5              (0, 7);
6          if (gestoreDati.cercaPrenotazioneTramiteCodice(
7              codicePrenotazioneNuovo) == null) {
8              codiceOccupato = false;
9          }
10     }
```

Il codice della prenotazione, al contrario, viene utilizzato direttamente dal bigliettaio e digitato a mano: un UUID di 36 caratteri quindi sarebbe inutilizzabile ed estremamente scomodo. Si troncano quindi i primi 7 caratteri, ottenendo un codice pratico.

Il troncamento però riduce lo spazio dei valori possibili e rende la collisione un evento più probabile. Per questo la generazione del codice è racchiusa in un ciclo che verifica l'assenza del codice fra quelli esistenti, ripetendo la generazione in caso di conflitto. La terminazione è garantita in probabilità, essendo lo spazio dei codici molto più ampio del numero di prenotazioni attese.

7.3 Ricerca e filtraggio

Le ricerche per identificativo `cercaUtente`, `cercaPrenotazioneTramiteCodice`, `cercaProiezioneTramiteCo` sono realizzate con scansione lineare e confronto tramite `equalsIgnoreCase`, restituendo `null` in assenza di corrispondenza.

L'insensibilità alle maiuscole nel confronto dei codici è una scelta di usabilità: il bigliettaio che digita un codice non deve preoccuparsi della forma dei caratteri.

Filtro delle proiezioni prenotabili. Il metodo `cercaProiezioniPrenotabili` combina due livelli di granularità:

```
1      public List<Proiezione> cercaProiezioniPrenotabili(String titolo, Genere
2          genere, LocalDate dataFine, Double prezzoMassimo, Double
3          prezzoMinimo) {
4          LocalDateTime adesso = LocalDateTime.now();
5          return cercaProiezione(titolo, genere, adesso.toLocalDate(),
6              dataFine, prezzoMassimo, prezzoMinimo).stream()
7              .filter(proiezione -> proiezione.getInizioProiezione().isAfter(
8                  adesso))
9              .toList();
10     }
```

Il metodo `cercaProiezione` filtra a granularità di giorno, poiché i suoi parametri sono di tipo `LocalDate`; cioè lascerebbe passare una proiezione odierna già conclusa. Il filtro aggiuntivo a granularità di istante elimina questo caso, garantendo che l'elenco mostrato al cliente contenga esclusivamente proiezioni effettivamente prenotabili. In questo modo l'insieme visualizzato coincide con questo accettato dal gestore, evitando che l'utente selezioni un'opzione destinata a essere rifiutata.

L'istante corrente è estratto in una variabile locale anziché essere invocato dentro il predicato:

oltre a soddisfare il requisito di variabile effettivamente finale imposto dalle lambda, garantisce che tutte le proiezioni siano confrontate con il medesimo riferimento temporale.

Capitolo 8

Sicurezza e gestione delle credenziali

La classe `Sicurezza` è dichiarata `final` con costruttore privato poiché è una classe di utilità e non deve essere né istanziata né estesa.

Algoritmo adottato

Parametro	Valore	Motivazione
Algoritmo	PBKDF2WithHmacSHA256	Funzione di derivazione di chiave progettata per l'hashing di password, disponibile nella libreria standard
Lunghezza salt	16 byte	Valore raccomandato; generato con <code>SecureRandom</code>
Lunghezza output	256 bit	Coerente con la funzione di hash
Iterazioni	620.000	Raccomandazione OWASP per l'algoritmo scelto

Si è deliberatamente evitato l'uso di funzioni di hash generiche come SHA-256 applicate direttamente alla password. Tali funzioni sono progettate per essere veloci, caratteristica che le rende inadatte allo scopo: un attaccante in possesso del file degli hash potrebbe testare miliardi di combinazioni al secondo. PBKDF2 introduce invece un fattore di lavoro configurabile che rende l'operazione deliberatamente costosa. **Salt casuale per utente.** Ogni utente ha un proprio salt generato con `SecureRandom`. Questo impedisce due categorie di attacco: l'uso di rainbow table, inefficaci quando ogni password è combinata con un valore casuale distinto, e l'identificazione di utenti con la stessa password, che senza salt produrrebbero hash identici. **Confronto a tempo costante**

```
1 public static boolean verificaLogin(char[] passwordInChiaro, byte[] salt, byte
   [] hashPrevisto, int iterazioniPreviste) {
2     byte[] passwordHash = hashPassword(passwordInChiaro, salt,
       iterazioniPreviste);
3     return MessageDigest.isEqual(passwordHash, hashPrevisto);
4 }
```

`MessageDigest.isEqual` confronta gli array in tempo indipendente dal numero di byte coincidenti. Un confronto ingenuo terminerebbe alla prima differenza, rendendo il tempo di esecuzione una funzione del numero di byte corretti e aprendo la strada a un attacco basato sulla misurazione dei tempi di risposta.

Uso di `char[]` anziché `String`. Le password in chiaro sono trattate come array di caratteri lungo tutto il percorso, dalla lettura dell'input fino al calcolo dell'hash. La motivazione è che le stringhe Java sono immutabili e permangono in memoria fino a quando il garbage collector interviene, senza possibilità di cancellazione esplicita. Un array invece è sovrascrivibile, ed è quanto viene fatto in `InterfacciaTerminale` al termine di ogni operazione che coinvolge una password:

```

1      finally {
2          if (password != null) {
3              Arrays.fill(password, '\0');
4          }
5      }

```

Il blocco finally garantisce la cancellazione anche in caso di eccezione. Coerentemente, `Sicurezza.hashPassword` invoca `spec.clearPassword()` nel proprio blocco finally, eliminando la copia interna mantenuta da `PBEKeySpec`.

Lettura non visibile delle password.

```

1  if (System.console() != null) {
2      password = System.console().readPassword();
3  } else {
4      password = scanner.nextLine().trim().toCharArray();
5  }

```

Quando è disponibile una console di sistema, la password viene letta senza essere visualizzata a schermo. Il ramo alternativo è necessario perché `System.console()` restituisce `null` quando l'applicazione è eseguita all'interno di un IDE, in tal caso la digitazione risulta visibile.

Aggiornamento automatico delle iterazioni. Il numero di iterazioni è memorizzato insieme a ciascun hash anziché essere una costante globale. Questo consente di innalzare il parametro nel tempo, come richiesto dall'evoluzione delle capacità di calcolo, senza invalidare le credenziali esistenti: al primo accesso successivo, l'utente vede la propria password ricalcolata con i nuovi parametri.

```

1      int iterazioniPreviste = utenteCorrente.getIterazioniHashing();
2      if (Sicurezza.verificaLogin(passwordInChiaro, Sicurezza.daBase64(salt),
3          Sicurezza.daBase64(passwordHashPrevista), iterazioniPreviste)) {
4          int iterazioniNuove = Sicurezza.getIterazioni();
5          if (iterazioniPreviste != iterazioniNuove) {
6              byte[] nuovoSalt = Sicurezza.generaSalt();
7              try {
8                  byte[] nuovaHash = Sicurezza.hashPassword(
9                      passwordInChiaro, nuovoSalt, iterazioniNuove
10                 );
11                  utenteCorrente.setPasswordHash(Sicurezza.aBase64(
12                      nuovaHash));
13                  utenteCorrente.setSaltBase64(Sicurezza.aBase64(
14                      nuovoSalt));
15                  utenteCorrente.setIterazioniHashing(
16                      iterazioniNuove);
17              } catch (IllegalArgumentException |
18                  IllegalStateException e) {
19                  utenteCorrente.setPasswordHash(
20                      passwordHashPrevista);
21                  utenteCorrente.setSaltBase64(salt);
22                  utenteCorrente.setIterazioniHashing(
23                      iterazioniPreviste);
24                  throw e;
25              }
26          }
27          gestoreDati.salvaUtentiSuCSV();
28      }

```

Il blocco try-catch implementa un ripristino se una delle tre assegnazioni fallisce. I valori precedenti vengono riscritti prima di propagare l'eccezione. Senza questa protezione l'utente resterebbe con credenziali parzialmente aggiornate e quindi non più verificabili.

Lo stesso schema di ripristino è applicato in `modificaDatiPersonali`, dove più campi vengono modificati in sequenza e il salvataggio avviene solo al termine.

Codifica per la persistenza. Hash e salt sono sequenze binarie e non sono rappresentabili direttamente in un file di test. Vengono quindi codificati in Base64 tramite **Sicurezza.aBase64** e decodificati con **daBase64**. Base64 produce un alfabeto che non include la virgola, risultando compatibile con il formato CSV senza necessità di protezione.

Capitolo 9

Gestione degli errori e delle eccezioni

La gestione degli errori è uniforme in tutta l'applicazione: ogni anomalia viene rilevata e la sua gestione spostata nel punto in cui è possibile deciderne le conseguenze. Il livello del modello e quello della logica rilevano e segnalano; il livello della presentazione decide come comunicarlo all'utente.

9.1 Gerarchia delle eccezioni

Sono state poi definite sei eccezioni, tutte estensioni di `RuntimeException`:

Eccezione	Condizione
<code>PermessiMancantiException</code>	Ruolo non autorizzato all'operazione
<code>PostiEsauritiException</code>	Capienza superata
<code>EtaMinimaConsentitaException</code>	Età minima non soddisfatta
<code>PrenotazioneInesistenteException</code>	Codice prenotazione non trovato
<code>SovrapposizioneProiezioneException</code>	Conflitto temporale nel palinsesto
<code>ProiezioneConPrenotazioniException</code>	Modifica/eliminazione bloccata

La scelta di eccezioni non controllate è perché queste eccezioni vengono sollevate a causa di input dell'utente. La loro gestione avviene in un unico punto, il livello di presentazione, dove vengono tradotte in messaggi comprensibili per l'utente.

L'introduzione di tipi specifici, in luogo del solo `IllegalArgumentException`, consente all'interfaccia di distinguere le cause e formulare messaggi mirati.

Per le violazioni di validità dei singoli attributi si è mantenuto `IllegalArgumentException`, in quanto appropriato e già parte della libreria standard.

9.2 Strategia di caricamento

I file CSV sono modificabili manualmente e possono quindi contenere righe anomale. I tre metodi di caricamento adottano quindi una strategia semplice, cioè una riga non valida non deve impedire il caricamento delle altre.

```
1      String[] datiPrenotazione = separaRigaCSV(linea, false);
2      try {
3          String codiceUnivoco = datiPrenotazione[0];
4          String usernameUtente = datiPrenotazione[1];
5          String codiceProiezione = datiPrenotazione[2];
6          int numeroBiglietti = Integer.parseInt(datiPrenotazione[3]);
7          Prenotazione prenotazione = new Prenotazione(codiceUnivoco,
8              usernameUtente, codiceProiezione, numeroBiglietti);
9          aggiungiPrenotazione(prenotazione);
10     } catch (IllegalArgumentException | IndexOutOfBoundsException e) {
```

```

10         System.err.println("Errore di validazione nella riga del CSV ("
11                               + linea + "): " + e.getMessage());
12     } catch (Exception e) {
13         System.err.println("Errore generico nella riga del CSV (" +
14                               linea + "): " + e.getMessage());
15     }

```

E' essenziale che l'intero blocco di parsing sia racchiuso nel `try-catch`, e non soltanto la costruzione dell'oggetto. Sia le conversioni numeriche (`NumberFormatException`) sia gli accessi all'array (`ArrayIndexOutOfBoundsException`) possono fallire su una riga malformata: collocarli al di fuori del blocco `try-catch` propagherebbe l'eccezione fino al costruttore di `GestoreDati`, impedendo l'avvio dell'applicazione per semplicemente una riga sbagliata.

La riga (o righe) problematiche vengono riportate integralmente tramite `System.err` insieme alla causa.

9.3 Propagazione degli errori di salvataggio

I metodi di salvataggio invece adottano una metodologia opposta a quella di caricamento:

```

1     } catch (IOException e) {
2         throw new IllegalStateException("Errore durante il salvataggio del file
3         proiezioni.csv", e);
4     }

```

Questo è fatto appositamente per un motivo. Un errore in lettura è ancora salvabile: si prosegue con i dati disponibili. Un errore in scrittura non lo è, perché significa che l'operazione compiuta dall'utente non è stata registrata nel file. Segnarlo solo su `System.err` impedirebbe all'utente di sapere che ci sia stato effettivamente un errore, dato che vedrebbe il messaggio di successo quando in realtà il dato è andato perduto.

9.4 Riferimenti orfani

Poiché le prenotazioni riferiscono utenti e proiezioni tramite identificativo, un file manipolato manualmente può contenere riferimenti non risolvibili. Nei metodi che operano su un singolo oggetto la condizione è segnalata come errore tramite `Validatore.validaOggetto`; nelle ricerche filtrate, dove l'obiettivo è produrre un risultato utilizzabile, i riferimenti non risolvibili sono semplicemente esclusi, così che una singola prenotazione orfana non comprometta l'intera ricerca.

Capitolo 10

Regole di dominio implementate

Questo è un riassunto di tutte le regole applicate, con l'indicazione breve dell'implementazione.

10.0.1 Regole - Proiezioni

Regola	Implementazione
Solo un proiezionista può gestire il palinsesto	GestoreProiezioni, verifica isProiezionista()
Non si programma proiezioni nel passato	GestoreProiezioni, confronto con LocalDateTime.now()
Nessuna sovrapposizione temporale	controllaSovrapposizione
La fine deve consentire la visione integrale	Proiezione.validaCoerenzaOrari
Modifica ed eliminazione bloccate in presenza di prenotazioni	haPrenotazioni

10.0.2 Regole - Prenotazioni

Regola	Implementazione
Solo un cliente registrato può prenotare	GestorePrenotazioni, verifica isCliente()
Si prenota solo per proiezioni non ancora iniziate	confronto con LocalDateTime.now()
I posti richiesti superano quelli liberi	controllo esplicito prima della creazione
rispetto dell'età minima del film	possoVisionareFilm
Ciascuno gestisce solo le proprie prenotazioni	confronto sullo username
Spostamento consentito solo fra giorni futuri	confronto su LocalDate
Annullamento consentito fino all'inizio dello spettacolo	confronto su LocalDateTime
Il bigliettaio consulta tutte le prenotazioni	verifica isBigliettaio()

10.0.3 Regole - Utenti

Regola	Implementazione
Username univoco	verifica in registraCliente e in aggiungiUtente
Username minuscolo alfanumerico	Utente.validaUsername
Password mai memorizzata in chiaro	Sicurezza
Data di nascita facoltativa	doppio costruttore in Utente

Piccola nota sulle regole temporali delle prenotazioni. La modifica di una prenotazione per uno spettacolo dello stesso giorno in cui si fa la modifica, non è possibile. Per

la creazione e l'annullamento invece si é adottato il confronto su `LocalDateTime`, consentendo annullamento e creazione fino all'inizio dello spettacolo.

Ulteriore nota sulla verifica dell'unicità dello `username`. Il controllo è presente in due punti: in `registraCliente`, dove produce un messaggio specifico per l'utente e in `GestoreDati.aggiungiUtente`, dove agisce come invariante della collezione. La duplicazione è intenzionale: il secondo controllo protegge anche il percorso di caricamento da file, dove un `utenti.csv` con due righe omonime produrrebbe altrimenti due utenti indistinguibili, di cui il secondo irraggiungibile.

Capitolo 11

Pattern e principi di progettazione adottati

11.1 Separazione in livelli

Descritta nel paragrafo 3.1. Ciascun livello dipende solo da quelli sottostanti.

11.2 Facade

I tre gestori costituiscono la facciata attraverso cui l'interfaccia accede al dominio. `InterfacciaTerminale` non manipola mai direttamente le collezioni né invoca i metodi di persistenza: ogni operazione passa per un metodo di gestore che ne verifica la legittimità.

Questo garantisce che nessuna regola possa essere aggirata modificando il livello di presentazione.

11.3 Dependency injection tramite costruttore

I gestori ricevono l'istanza di `GestoreDati` come parametro del costruttore, anziché crearla internamente o accedervi attraverso una variabile globale.

Il riferimento è dichiarato `final`, garantendo che non possa essere sostituito dopo la costruzione. La condivisione della medesima istanza fra i tre gestori è ciò che rende consistente la vista sui dati.

11.4 Classi di utilità

`Sicurezza` e `Validatore` sono dichiarate `final` con costruttore privato ed espongono soltanto metodi statici. Il costruttore privato impedisce sia l'istanziamento, priva di senso per una classe senza stato sia l'estensione grazie a `final`.

11.5 Overloading per la leggibilità della validazione

`Validatore` espone un metodo generico e una serie di overloader tipizzati:

```
1      public static void validaOggetto(Object oggetto, String messaggioErrore
2          ) {
3          if (oggetto == null) {
4              throw new IllegalArgumentException(messaggioErrore);
5          }
6      }
7      public static void validaOggetto(Utente utente) {
```

```

8         validaOggetto(utente, "L'utente non può essere nullo.");
9     }
10
11     public static void validaOggetto(Proiezione proiezione) {
12         validaOggetto(proiezione, "La proiezione non può essere nulla.")
13         ;
14     }
15     ...

```

Gli overload delegano tutti all'implementazione generica, aggiungendo il messaggio appropriato al tipo. Il chiamante scrive `Validatore.validaOggetto(utente)` e ottiene automaticamente un messaggio d'errore pertinente, senza doverlo ripetere in ogni punto di chiamata. La risoluzione avviene a tempo di compilazione sulla base del tipo statico dell'argomento.

Capitolo 12

Limiti della soluzione sviluppata

Assenza di concorrenza

Il sistema presuppone un solo utilizzatore alla volta. Nessuna struttura dati è sincronizzata e la strategia di salvataggio riscrive integralmente il file a ogni operazione: quindi due istanze concorrenti sui medesimi file produrrebbero perdita di aggiornamenti.

Riscrittura integrale in salvataggio

Ogni modifica comporta la riscrittura completa del file interessato. La soluzione è semplice ma è algoritmicamente inefficiente, dato che ha costo lineare sul numero di record. Su volumi superiori avrebbe problemi.

Password visibile in ambiente di sviluppo

Quando il programma viene avviato in un IDE, l'inserimento della password non è protetto come lo sarebbe invece in un terminale e quindi risulterebbe in chiaro. In questo ramo, la password inoltre transita per un oggetto di tipo `String` intermedio, non cancellabile dalla memoria.

Capitolo 13

Sitografia e bibliografia

- OWASP Foundation, Password Storage Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- Oracle, Java Platform, API
<https://docs.oracle.com/en/java/javase/21/>
- Regex101, Regex builder and tester
<https://regex101.com/>
- Wikipedia, ISO 8601, Convenzioni per le date
https://en.wikipedia.org/wiki/ISO_8601
- IETF, RFC 8018, Crittografia
<https://www.rfc-editor.org/info/rfc8018/>
- IETF, RFC 4180, Formati CSV
<https://www.rfc-editor.org/info/rfc4180/>